

SIMATS ENGINEERING ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 30%

Duration : 45 Minutes

QUESTIONS: 2 **Total Marks:** 20

Annexure A

Descriptive Written Test

Code of Conduct:

I Senthamizhselvan K 192321143 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Senthamizhselvan K

Annexure A

Descriptive Written Test

1. A smart home automation system is being designed to manage lighting, temperature, and security based on user behavior and environmental conditions. The system must respond to real-time inputs such as motion detection, temperature changes, and time of day while also learning user preferences over time. In this context, explain the different types of intelligent agents (simple reflex, model-based, goal-based, and utility-based agents) and analyze how each type would function within this system. Justify which type of agent or hybrid approach would be most effective for ensuring comfort, energy efficiency, and security.

Answer:

1. Introduction – the smart home as an intelligent agent

An **intelligent agent** perceives its environment through sensors and acts on it through actuators so as to maximise a performance measure. The smart home is such an agent:

PEAS	Smart home automation system
Performance measure	User comfort (right light and temperature), low energy use/cost, security (no intrusion, fast alarm), safety.
Environment	House, occupants, weather, time of day, electricity tariff – partially observable, stochastic (people behave unpredictably), dynamic, sequential, continuous.
Actuators	Lights, heater/AC, fans, door locks, alarm/siren, notifications, smart plugs.
Sensors	Motion (PIR), temperature/humidity, light level, clock, door/window contacts, cameras, smoke sensor, smart meter.

2. Types of intelligent agents and how each works in the smart home

(a) Simple reflex agent. It selects actions using *only the current percept* through condition–action rules (if–then), with no memory of the past: *action = rule(percept)*.

- *In the smart home:* IF motion detected AND light level low THEN lights ON; IF temperature > 30 °C THEN AC ON; IF door/window opens while system is armed THEN sound alarm; IF smoke detected THEN unlock doors and alert.
- *Advantages:* very fast, simple, cheap, predictable and easy to verify – ideal for safety and security emergencies.
- *Limitations:* no memory and no model, so it works only if the environment is fully observable from the current percept. It switches lights off when a person sits still (no motion), cannot tell “nobody home” from “person sleeping”, may toggle the AC on and off repeatedly near the threshold, and cannot learn or plan.

(b) Model-based reflex agent. It keeps an *internal state* that tracks the part of the world it cannot see, using a model of *how the world evolves* and *how its actions affect it*; rules are then applied to this state.

- *In the smart home:* state = which rooms are occupied, time since last motion, door/lock status, current and predicted room temperature, which appliances are on. Rules such as “IF the room was occupied and the last motion was less than 10 minutes ago THEN keep the lights on” or “IF the temperature is rising 1 °C per hour because of sunshine THEN start cooling early”.
- *Advantages:* handles partial observability (a sleeping person, a closed door), avoids flickering behaviour, can detect unusual patterns (door opened while all occupants are known to be inside).
- *Limitations:* still follows fixed rules – it does not know *why* it acts, cannot plan ahead or weigh trade-offs.

(c) Goal-based agent. It has explicit *goals* and uses *search and planning* to choose a sequence of actions that achieves them: it asks “what will happen if I do this, and will it get me to my goal?”.

- *In the smart home*: goals such as “house at 22 °C when the user returns at 6 p.m.”, “all doors locked and lights off by 11 p.m.”, “hot water ready at 6 a.m.”. The agent plans when to start heating/cooling, which sequence of lights to turn off, or how to react if a window is open when the goal is “secure house”.
- *Advantages*: flexible – changing the goal changes the behaviour with no rewriting of rules; anticipates the future.
- *Limitations*: a goal is only “achieved / not achieved”; it cannot say which of two ways of reaching the goal is better (cheaper, more comfortable), and it can be slow when planning is expensive.

(d) Utility-based agent. It uses a *utility function* that maps each state to a number measuring “how happy” the agent is, and selects the action with the highest *expected utility*, which allows trade-offs between conflicting goals and handling uncertainty.

- *In the smart home*: $U = w_1 \cdot \text{comfort} - w_2 \cdot \text{energy cost} - w_3 \cdot \text{security risk}$. For example, if the tariff is high, the agent accepts 1 °C less comfort to save energy; if the user is probably arriving soon (probability 0.8), it pre-cools; in a high-risk situation (night, nobody home), it gives security the highest weight.
- *Advantages*: best decision quality, deals with conflicting objectives (comfort versus energy) and uncertainty (when the user returns).
- *Limitations*: the utility function must be designed or learned, and computing expected utility is more costly.

(e) Learning agent (needed for “learning user preferences over time”). Any of the above can be extended with a learning element, a critic, a performance element and a problem generator: the critic uses feedback (the user manually changes the thermostat, turns a light back on, or energy bills) to improve the rules, goals or the utility weights – so the home learns, for example, that the user likes 24 °C in the evening and 21 °C when sleeping.

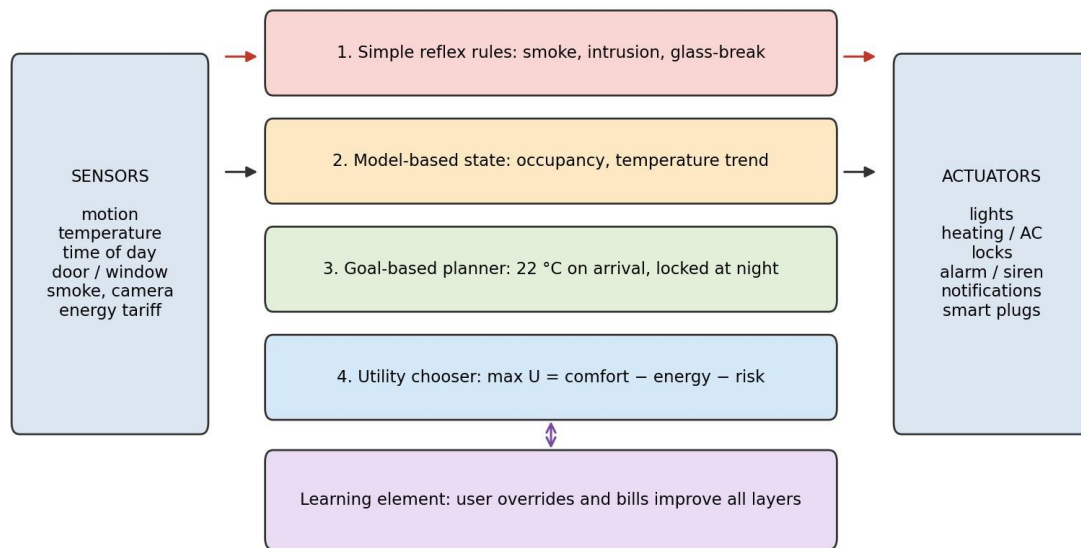
3. Comparison

Agent type	Uses	Handles partial observability	Plans ahead	Trade-offs	Smart-home role
Simple reflex	Current percept + rules	No	No	No	Emergency / security reactions
Model-based	Internal state + model	Yes	No	No	Occupancy, temperature tracking
Goal-based	State + goals + search	Yes	Yes	No	Schedules (arrival, bedtime)
Utility-based	State + utility function	Yes	Yes	Yes	Comfort vs energy vs security
Learning	Feedback + critic	Yes	Yes	Yes	Personalisation over time

4. Justification – the most effective approach is a hybrid (layered) agent

No single agent type is enough: the system must *react in milliseconds* (security), *remember* what it cannot see (occupancy), *plan* (arrival time), *trade off* objectives (comfort versus energy) and *learn* preferences. The most effective design is therefore a **hybrid agent with a utility-based decision core, built on a model-based state and protected by a reflex safety layer, and improved by a learning element**:

Proposed hybrid (layered) agent for the smart home



- **Security:** simple reflex rules at the top priority (smoke, intrusion, glass-break, door forced) give an immediate response that cannot be delayed by planning; the model-based state reduces false alarms (e.g., a known occupant arriving).
- **Comfort:** the model-based state (who is in which room, temperature trend) and the goal-based planner (pre-heat/cool for the arrival time) provide comfort without waiting for the room to become uncomfortable; learning adapts the set-points to each user.
- **Energy efficiency:** the utility-based chooser balances comfort against cost (peak tariff, empty rooms, standby appliances), switching off devices only when the model shows no one is there and choosing the cheapest way to achieve the target temperature.
- **Learning:** the system improves from user overrides and electricity bills, so rules, goals and utility weights adapt over time.

Conclusion: simple reflex agents give speed and safety, model-based agents give awareness, goal-based agents give foresight, and utility-based agents give optimal trade-offs. Combining them with learning produces a smart home that is comfortable, energy-efficient and secure.

2. A robot is placed in an unknown maze and must find a path to the exit without prior knowledge of the layout. Explain how uninformed search strategies like Uniform Cost Search (UCS) and Iterative Deepening Search (IDS) can help solve this problem.

Discuss the advantages and disadvantages of these algorithms in terms of time and space complexity. Relate the problem to different types of intelligent agents and explain how agent design affects decision-making. Suggest which combination of agent type and search strategy would be most effective and justify your choice.

Answer:

1. Problem formulation

- **States:** the robot's position in the maze (cell/junction). **Initial state:** the starting cell. **Goal test:** the cell is the exit. **Actions:** move forward / left / right / back to an adjacent open cell. **Path cost:** distance, time or energy of each move (1 per move in a plain grid).
- Because the layout is unknown, the robot can see only nearby cells (partially observable). It builds a map as it explores, and every newly discovered open cell is added to the frontier. An **uninformed (blind) search** is suitable because there is no information about where the exit is, so no heuristic can be used: the strategy only uses the problem definition (branching factor b is small in a maze, usually 1–3, depth d = number of steps to the exit, m = maximum depth).

2. Uniform Cost Search (UCS)

- **How it works:** keeps a priority queue ordered by path cost $g(n)$ and always expands the cheapest frontier node, so paths are explored in order of increasing total cost (a “contour” of equal cost spreads out from the start). When the exit is selected for expansion, its path is the cheapest. With equal step costs it behaves like BFS.
- **Use in the maze:** when the corridors have different costs (long corridors, muddy or slippery floor, ramps, battery use), UCS finds the minimum-cost route, not just the one with fewest turns. Example: corridor A has 3 muddy cells of cost 3 each (total 9); corridor B has 5 normal cells of cost 1 (total 5). UCS selects B, whereas BFS or IDS would select A.
- **Properties:** complete if every step cost $\geq \epsilon > 0$ and b is finite; optimal. **Time and space:** $O(b^{1+C^*/\epsilon})$, where C^* is the cost of the optimal solution (this equals $O(b^d)$ for equal costs).
- **Advantages:** optimal for any positive step costs; handles weighted mazes; no heuristic needed.
- **Disadvantages:** must store the entire frontier and explored set, so memory grows exponentially (a problem for a robot with limited memory); explores in all directions with no sense of direction toward the exit; may expand many cheap but useless corridors before reaching a costly but correct one; zero-cost or tiny-cost steps cause long delays; in a real, physical maze the robot must travel back to the next cheapest frontier node, which wastes energy.

3. Iterative Deepening Search (IDS)

- **How it works:** runs depth-limited DFS repeatedly with limits 0, 1, 2, 3, ... until the exit is found. It combines DFS's small memory with BFS's completeness and shallowest-solution guarantee.
- **Use in the maze:** the robot explores corridors to the current depth limit, then backtracks and increases the limit. A walking robot only needs to remember the current path (plus visited cells for cycle checking), which suits it, and the exit at the smallest number of moves is found.
- **Properties:** complete (finite b); optimal when all step costs are equal (finds the shallowest goal). **Time** $O(b^d)$; **space** $O(bd)$ ($O(d)$ in a maze with small b).
- **Cost of repetition is small:** most nodes are at the deepest level. With $b = 10$ and $d = 5$, IDS generates $50 + 400 + 3,000 + 20,000 + 100,000 = 123,450$ nodes, whereas BFS generates about 1,111,100 – so the repeated work is only about 11%.

- **Advantages:** very low memory, complete, optimal for uniform cost, no need to know the depth of the exit in advance, works well when the search space is large and the exit depth is unknown.
- **Disadvantages:** regenerates shallow nodes in every iteration; it ignores path cost (not optimal when step costs differ); mazes contain loops, so without cycle checking it wastes time revisiting cells; the robot physically retraces its path at every iteration, which is costly in real movement.

4. Comparison of UCS and IDS

Criterion	Uniform Cost Search (UCS)	Iterative Deepening Search (IDS)
Data structure	Priority queue (by $g(n)$)	Stack / recursion (depth-limited DFS)
Complete?	Yes (step cost $\geq \epsilon > 0$)	Yes (finite branching factor)
Optimal?	Yes – for any positive costs	Yes only for equal step costs (shallowest goal)
Time complexity	$O(b^{1+C^*/\epsilon})$	$O(b^d)$
Space complexity	$O(b^{1+C^*/\epsilon})$ – exponential	$O(bd)$ – linear
Best when	Step costs differ; memory is available	Memory is limited; step costs are equal; depth of exit is unknown
Main weakness	Memory and expanding all cheaper nodes	Repeated work; ignores costs

5. Relation to types of intelligent agents; effect of agent design on decisions

Agent type	Behaviour in the maze	Effect on decision-making
Simple reflex	Rules based on the current view only, e.g. “follow the right wall”, “turn if blocked”.	Fast but has no memory of visited cells; it can loop forever in mazes containing loops and cannot guarantee finding the exit or the shortest path. No search.
Model-based	Keeps a map of visited cells, walls and junctions and its own position (via odometry).	Needed because the maze is unknown and only partly visible; it can avoid revisiting cells and supports DFS/IDS/UCS on the learned map.
Goal-based	Goal = reach the exit; searches/plans on its model with BFS, IDS, UCS.	Chooses actions by looking ahead; it finds a path to the exit, and re-plans when a new wall is discovered.
Utility-based	Evaluates paths using a utility = $-(\text{distance, energy, risk of dead ends or hazards})$.	Chooses the best among several paths: uses UCS-like cost reasoning and can compromise between speed, battery use and safety.
Learning	Learns maze features and good heuristics from previous attempts.	Improves later runs (remembers the exit position, dead ends) and enables informed search such as A* in the future.

Agent design determines decision quality: a reflex design reacts instantly but cannot reason about the unseen parts of the maze; a model-based design builds knowledge that makes search possible; a goal-based design turns that knowledge into a plan; a utility-based design chooses the *best* plan when costs differ. The search strategy is the “brain” of the planning component of such an agent.

6. Recommended combination and justification

Most effective choice: a model-based, goal-based agent (with a utility-aware cost function) that uses IDS for exploration of the unknown maze and UCS on the map it has built when step costs differ.

- **Model-based** – the maze is unknown and partially observable, so the robot must keep a map and position to avoid loops and to re-plan when new walls are sensed.
- **Goal-based** – the objective is a single goal (the exit), which fits search-based planning.
- **IDS for exploration** – no prior knowledge of the exit’s depth, memory is limited ($O(bd)$), it is complete, and with uniform move costs it finds the shortest route; the small overhead of repetition is acceptable (about 11% in the example).

- **UCS when costs vary** – if corridors have different costs (mud, slopes, energy), IDS is no longer optimal; UCS on the discovered map returns the cheapest route (and a simple reflex layer for collision avoidance stays active for safety).
- **Why not the alternatives:** a pure reflex agent can loop; BFS and UCS need exponential memory in large mazes; DFS alone is neither complete (in loops) nor optimal.

Conclusion: combining a model-based, goal-based agent with IDS (uniform cost, limited memory) and UCS (variable cost) gives a robot that is complete, memory-efficient and able to find the best path to the exit in an unknown maze.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand